DEVOPS WORKSHOP

IPv4 vs IPv6

Lab Workbook & Field Guide

A self-contained companion to the 60-minute workshop: a concept refresher, a hands-on containerlab + Nokia SR Linux lab, fill-in exercises, a DevOps gotchas cheat sheet, and a full answer key.

Audience	Aspiring DevOps engineers with basic networking knowledge
Duration	~20 min hands-on lab (within a 60-min session)
You need	Linux host with Docker + containerlab, and two SR Linux containers
Goal	Configure a dual-stack IPv6 link, route across it, and see why NAT isn't needed

NOTE: **How to use this workbook —** Sections 1–2 are a quick refresher you can skim before the lab or keep open during the talk. Section 3 is the lab — work top to bottom. Look for the ✎ marks: those are blanks to fill in as you go. Stuck? Section 6 has the full answer key. Lines starting with # in code blocks are comments, not commands.

1. The 5-minute refresher

Why IPv6 exists

- IPv4 has ~4.3 billion addresses (2^{32}) — fewer than there are people on Earth. The global free pool is exhausted.
- NAT and private ranges (RFC 1918) were rationing to stretch IPv4 — never security features. They cost you latency, overlapping CIDRs, egress bottlenecks, and obscured source IPs.
- IPv6 has $2^{128} \approx 3.4 \times 10^{38}$ addresses, so every device, container, and pod can hold a real, globally unique address — no sharing, no translation.

Reading an IPv6 address

128 bits, written as eight 16-bit blocks ("hextets") in hexadecimal, separated by colons. The first half is typically the network/prefix; the second half the interface/host.

```
Full:        2001:0db8:0000:0000:0000:8a2e:0370:7334
Abbreviated: 2001:db8::8a2e:370:7334
```

The two abbreviation rules

1. Drop leading zeros within each block: :0db8: → :db8:
2. Collapse ONE contiguous run of all-zero blocks to :: (only once per address, or it's ambiguous).

Exercise 1.1 — Shorten these (answers in Section 6)

✎ a) 2001:0db8:0000:0000:0000:0000:0000:0001 → _____

✎ b) fe80:0000:0000:0000:0202:b3ff:fe1e:8329 → _____

✎ c) 2001:0db8:0000:00a3:0000:0000:0000:1234 → _____

Address types you'll actually meet

Type	Range	Think of it as
Global Unicast	`2000::/3`	A public IP — and it lives on the host itself
Link-Local	`fe80::/10`	Auto-set on every interface; used by the plumbing (ND, routing)
Unique Local (ULA)	`fd00::/8`	The v6 'private' range — closest cousin to RFC 1918
Multicast	`ff00::/8`	Replaces broadcast entirely (there is no broadcast in v6)
Loopback	`::1/128`	localhost

Unspecified / default	`::` · `::/0`	`::` = bind-to-all (socket 'any'); `::/0` = default route (the prefix 'any', = 0/0)

How hosts get addresses

- `SLAAC` — the router advertises the /64 prefix and the host builds its own address. No DHCP needed.
- `DHCPv6` — stateful, central assignment when you want managed control and tracking.
- `NDP` (Neighbor Discovery) — replaces ARP, using ICMPv6 + multicast. You'll inspect its table in the lab.

WHY IT MATTERS: Host/LAN subnets are /64 because SLAAC uses the lower 64 bits as the interface ID. Providers hand you a /48 or /56 to carve into /64s, so per-LAN sizing math disappears. But /64 is a convention, not a law: point-to-point links commonly use /127 (RFC 6164) and loopbacks/single hosts use /128 — exactly like /31 and /32 in IPv4.

The DevOps impact

	IPv4	IPv6	Why you care
NAT	Required for private nodes	Not needed	No NAT gateways or CIDR overlap
Header	Variable, options	Fixed 40 bytes	Faster, predictable routing/firewalls
Subnet	VLSM math	Standard /64	Sizing leaves your IaC templates
IPsec	Optional add-on	Standardized*	Cleaner baseline (but not auto-on)

** IPsec was a MUST in early IPv6 specs but was relaxed to SHOULD in RFC 6434. It is standardized and consistently available, but NOT automatically enabled — don't repeat the "IPv6 is encrypted by default" myth.*

Security: routability ≠ reachability

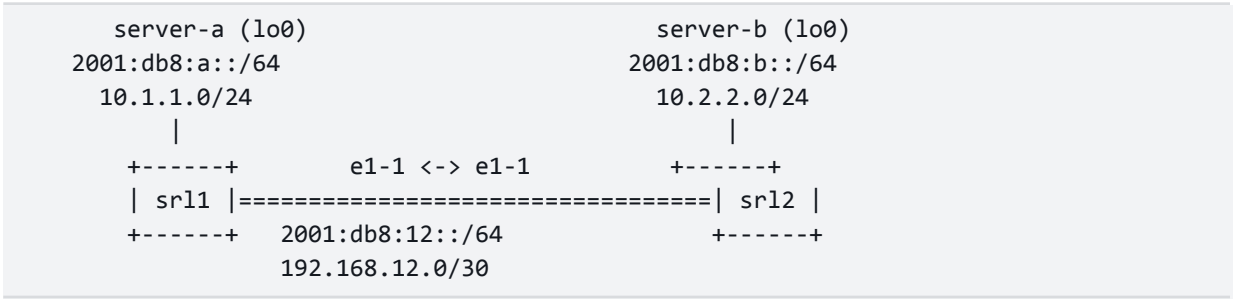
- A globally unique, routable address is NOT an open door. Reachability is a policy decision.
- The NAT fallacy: "behind NAT, nobody can reach me." NAT hid addresses as a side effect of rationing — it was never the control keeping you safe.
- Stateful firewalls / cloud Security Groups drop unsolicited inbound by default — in v4 AND v6. A public IPv6 host is invisible if the edge policy drops inbound.
- Scale helps too: a single /64 holds 18.4 quintillion addresses — scanning it at 1,000,000/sec takes ~580,000 years. Random host-discovery scanning is effectively dead.

WHY IT MATTERS: Want NAT's "outbound-only" behaviour without NAT? Use an Egress-Only Internet Gateway (AWS) or the equivalent egress firewall on Azure/GCP: it permits flows the instance starts and drops unsolicited inbound — no translation, no bottleneck, and the gateway itself is free on AWS.

2. The lab at a glance

You'll deploy two Nokia SR Linux routers connected back-to-back, bring the link up in BOTH address families (dual-stack), and add static routes so two "server" subnets can reach each other. Every step maps to a concept from the talk.

Topology



Addressing plan

Device	Interface	IPv6	IPv4	Role
srl1	`ethernet-1/1.0`	`2001:db8:12::1/64`	`192.168.12.1/30`	link to srl2
srl1	`lo0.0`	`2001:db8:a::1/64`	`10.1.1.1/24`	server-a subnet
srl2	`ethernet-1/1.0`	`2001:db8:12::2/64`	`192.168.12.2/30`	link to srl1
srl2	`lo0.0`	`2001:db8:b::1/64`	`10.2.2.1/24`	server-b subnet

NOTE: We use 2001:db8::/32 — the official documentation prefix (RFC 3849), so these addresses are safe to use in labs and slides and will never clash with real internet routes.

WATCH OUT: Sizing choice: this lab uses /64 everywhere for clarity. In production you'd typically make the srl1–srl2 point-to-point link a /127 (RFC 6164) and a router's own loopback a /128. The lo0 here is a /64 on purpose — it represents a LAN/server subnet behind the router, which legitimately is a /64.

Prerequisites

- A Linux host with Docker and containerlab installed (`containerlab version` to check).
- The SR Linux image pulled: `docker pull ghcr.io/nokia/srlinux:latest`
- The file `ipv6-lab.clab.yml` from this lab bundle in your working directory.

3. Hands-on lab

Step 0 — Deploy the topology

```
# from the folder containing ipv6-lab.clab.yml
sudo containerlab deploy -t ipv6-lab.clab.yml

# confirm both nodes are running
sudo containerlab inspect -t ipv6-lab.clab.yml
```

✎ How many nodes does inspect report running? _____

Step 1 — Connect to srl1

```
# open the SR Linux CLI on srl1
docker exec -it clab-ipv6-lab-srl1 sr_cli

# (repeat in a second terminal for srl2 when prompted)
# docker exec -it clab-ipv6-lab-srl2 sr_cli
```

NOTE: SR Linux uses a candidate/commit model: you stage changes with set ... then apply them with commit now. Nothing takes effect until you commit.

Step 2 — Configure the dual-stack link

On srl1, enter candidate mode and configure ethernet-1/1 with BOTH an IPv4 and an IPv6 address:

```
enter candidate

set / interface ethernet-1/1 admin-state enable
set / interface ethernet-1/1 subinterface 0 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv4 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv4 address 192.168.12.1/30
set / interface ethernet-1/1 subinterface 0 ipv6 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv6 address 2001:db8:12::1/64

# add the subinterface to the default routing table
set / network-instance default interface ethernet-1/1.0

commit now
```

Now do the mirror image on srl2 (note the ::2 / .2 addresses):

```
enter candidate
set / interface ethernet-1/1 admin-state enable
set / interface ethernet-1/1 subinterface 0 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv4 admin-state enable
```

```
set / interface ethernet-1/1 subinterface 0 ipv4 address 192.168.12.2/30
set / interface ethernet-1/1 subinterface 0 ipv6 admin-state enable
set / interface ethernet-1/1 subinterface 0 ipv6 address 2001:db8:12::2/64
set / network-instance default interface ethernet-1/1.0
commit now
```

WHY IT MATTERS: One subinterface carries both families at once. That IS dual-stack — exactly what cloud VPCs and Kubernetes pods do in production.

Step 3 — Verify the link & read the address

```
# on srl1, look at the subinterface
show interface ethernet-1/1.0

# ping srl2 across the link over IPv6
ping 2001:db8:12::2 network-instance default

# ...and over IPv4, to prove both run side by side
ping 192.168.12.2 network-instance default
```

✎ Did both pings succeed (y/n)? _____

Exercise 3.1 — Abbreviation in the wild

Look at the IPv6 address SR Linux prints in the show output. Write it exactly as displayed, then write its fully-expanded (no-shortcuts) form:

✎ As displayed: _____

✎ Fully expanded: _____

Step 4 — Inspect the neighbor table (ARP's replacement)

IPv6 has no ARP — Neighbor Discovery (NDP) does the job using ICMPv6 and multicast. After the ping, srl1 has learned srl2 as a neighbor:

```
show arpnd neighbors interface ethernet-1/1 subinterface 0
# (you can also browse:  info from state interface ethernet-1/1 subinterface 0 ipv6
neighbor-discovery)
```

✎ What link-local (fe80::...) or global address does srl1 list for its neighbor?

_____

NOTE: Notice the neighbor's link-local fe80:: address — it was auto-created with no configuration. That's SLAAC/ND plumbing working underneath your global addresses.

Step 5 — Add the 'server' subnets and route between them

Give each router a loopback that represents a server/pod subnet behind it, then add static routes so the two subnets can reach each other. On srl1:

```
enter candidate
# loopback = server-a subnet
set / interface lo0 subinterface 0 ipv4 admin-state enable
set / interface lo0 subinterface 0 ipv4 address 10.1.1.1/24
set / interface lo0 subinterface 0 ipv6 admin-state enable
set / interface lo0 subinterface 0 ipv6 address 2001:db8:a::1/64
set / network-instance default interface lo0.0

# static routes toward server-b (behind srl2)
set / network-instance default next-hop-groups group v6-srl2 nexthop 0 ip-address
2001:db8:12::2
set / network-instance default next-hop-groups group v4-srl2 nexthop 0 ip-address
192.168.12.2
set / network-instance default static-routes route 2001:db8:b::/64 next-hop-group
v6-srl2
set / network-instance default static-routes route 10.2.2.0/24 next-hop-group v4-srl2
commit now
```

On srl2 (loopback = server-b, routes point back toward server-a):

```
enter candidate
set / interface lo0 subinterface 0 ipv4 admin-state enable
set / interface lo0 subinterface 0 ipv4 address 10.2.2.1/24
set / interface lo0 subinterface 0 ipv6 admin-state enable
set / interface lo0 subinterface 0 ipv6 address 2001:db8:b::1/64
set / network-instance default interface lo0.0
set / network-instance default next-hop-groups group v6-srl1 nexthop 0 ip-address
2001:db8:12::1
set / network-instance default next-hop-groups group v4-srl1 nexthop 0 ip-address
192.168.12.1
set / network-instance default static-routes route 2001:db8:a::/64 next-hop-group
v6-srl1
set / network-instance default static-routes route 10.1.1.0/24 next-hop-group v4-srl1
commit now
```

Step 6 — Ping across the fabric

```
# on srl1: check the IPv6 route table for the new prefix
show network-instance default ipv6 route summary

# reach server-b's subnet across the routed link
ping 2001:db8:b::1 network-instance default

# same thing over IPv4
```

```
ping 10.2.2.1 network-instance default
```

- ✎ Does 2001:db8:b::1 reply? _____
- ✎ Does the route table show 2001:db8:b::/64 via 2001:db8:12::2?

_____

WHY IT MATTERS: You just routed IPv6 exactly the way you'd route IPv4 — same mental model, same route table, no NAT anywhere in the path.

Step 7 (Bonus) — Prove routable ≠ reachable

The remote subnet is routable. Now make it unreachable with policy, without changing any address. Apply an ACL on srl2 that drops inbound ICMPv6 echo, then re-ping from srl1.

```
# on srl2 — define an IPv6 ACL that drops ICMPv6 echo-request
enter candidate
set / acl acl-filter block-ping type ipv6 entry 10 match ipv6 next-header icmp6
set / acl acl-filter block-ping type ipv6 entry 10 match ipv6 icmp6 type echo-request
set / acl acl-filter block-ping type ipv6 entry 10 action drop
set / acl acl-filter block-ping type ipv6 entry 100 action accept

# attach it inbound on the link subinterface
set / acl interface ethernet-1/1.0 input acl-filter block-ping type ipv6 first
commit now
```

```
# back on srl1 — the address is still routed, but the ping now fails
ping 2001:db8:b::1 network-instance default
```

- ✎ Routable, but reachable? (the ping should now FAIL) _____

WHY IT MATTERS: The address never changed and the route is still in the table — only the policy at the edge changed. THAT is what a cloud Security Group does. Reachability is policy, not addressing — which is why losing NAT loses you nothing security-wise.

WATCH OUT: SR Linux ACL syntax shifted across releases. Newer images (24.3+) use a unified model: acl acl-filter block-ping type ipv6 entry 10 ... and acl interface ethernet-1/1.0 input acl-filter block-ping. If a command errors, run show version and check the ACL guide at documentation.nokia.com/srlinux for your release.

Step 8 — Clean up

```
# exit the SR Linux CLI (type 'quit'), then on the host:
sudo containerlab destroy -t ipv6-lab.clab.yml --cleanup
```

4. DevOps gotchas cheat sheet

The practical residue — the things that bite first when you start running dual-stack for real.

Gotcha	What to do
URLs need brackets	Wrap the address: `http://[2001:db8::1]:8080` — the colon clashes with the port.
Bind address	`::` means all IPv6 (often v4 too, via mapping). `0.0.0.0` is v4-only. Know which your service uses.
Localhost	The v6 loopback is `::1`, not `127.0.0.1`. Health checks and allowlists must include it.
Firewall BOTH families	Every v4 rule needs a v6 twin, or you leave a silent gap (open or unreachable).
Log & parse both	Normalize addresses before comparing; `::` and case make naive string/regex matching fail.
Happy Eyeballs	Clients race v4 vs v6 and pick the winner. A half-broken v6 path shows up as 'random slowness'.
DNS records	v6 uses `AAAA` records (v4 uses `A`). Missing AAAA = clients never try v6.
App / library readiness	Confirm your runtime, drivers, and libraries accept v6 literals before flipping a service.

5. Quick command reference

SR Linux

Task	Command
Enter / apply config	`enter candidate … commit now`
Show a subinterface	`show interface ethernet-1/1.0`
Ping (v6 or v4)	`ping <addr> network-instance default`
Neighbor (ND) table	`show arpnd neighbors interface ethernet-1/1.0`
IPv6 route table	`show network-instance default ipv6 route summary`
Save running config	`save startup` (tools system configuration save)

Linux host (handy for v6 in general)

Task	Command
Show v6 addresses	`ip -6 addr`

Show v6 routes	`ip -6 route`
Show neighbors (ARP/ND)	`ip -6 neigh`
Ping v6	`ping -6 2001:db8::1` (or `ping6`)
Curl a v6 literal	`curl -g 'http://[2001:db8::1]:8080'`

6. Answer key

Exercise 1.1 — Abbreviation

Full	Abbreviated
`2001:0db8:0000:0000:0000:0000:0000:0001`	`2001:db8::1`
`fe80:0000:0000:0000:0202:b3ff:fe1e:8329`	`fe80::202:b3ff:fe1e:8329`
`2001:0db8:0000:00a3:0000:0000:0000:1234`	`2001:db8:0:a3::1234`

Note for (c): the :: replaces the longest run of zero blocks (the last three), and the single 0 block between db8 and a3 is written as a literal 0 — you can only use :: once.

Lab checkpoints

- Step 0: inspect should show 2 running nodes (clab-ipv6-lab-srl1 and -srl2).
- Step 3: both the IPv6 (2001:db8:12::2) and IPv4 (192.168.12.2) pings succeed once the link is committed on both routers.
- Exercise 3.1: SR Linux displays the abbreviated form, e.g. `2001:db8:12::1`. Fully expanded, srl1's link address is `2001:0db8:0012:0000:0000:0000:0000:0001` (note the 3rd block is 0012, and the :: stands for the four all-zero blocks).
- Step 4: srl1 lists srl2's neighbor — both a link-local fe80::/10 address (auto-generated) and, after Step 5, reachability to its global addresses.
- Step 6: 2001:db8:b::1 replies; the route table shows 2001:db8:b::/64 resolved via next-hop 2001:db8:12::2. The IPv4 ping to 10.2.2.1 succeeds the same way.
- Step 7 (bonus): after the ACL is applied on srl2, the ping from srl1 FAILS even though the route is still present — routable, but no longer reachable.

WATCH OUT: If a ping fails: confirm you ran commit now on BOTH routers, that admin-state is enable on the interface AND subinterface, and that the subinterface was added to network-instance default. 90% of lab issues are a missing commit or a missing network-instance binding.

Instructor note

The files srl1.cli and srl2.cli in the lab bundle contain the complete, correct configuration for each router. You can either let students type the steps (recommended) or pre-load these via the topology's startup-config option to demo the finished state first.